

UNIVERSIDAD AUTÓNOMA DEL ESTADO DE HIDALGO

Instituto de Ciencias Básicas e Ingeniería

Centro de Investigación en Tecnologías de Información y Sistemas

Licenciatura en Ciencias Computacionales

Criptografía (Optativa II)

Octavo semestre

Reporte de Tarea 05

“ANÁLISIS DE MINIDES Y ATAQUE DE FUERZA BRUTA”

Trabajo realizado por:

Carlos Alberto Lara Hernández

Profesor: Dr. Virgilio López Morales

Resumen

En este trabajo se realizó el análisis del algoritmo MiniDES, un cifrado de bloques educativo con clave de 10 bits. Se implementó un ataque de fuerza bruta capaz de recuperar la clave en menos de un segundo. Se evaluó la escalabilidad del ataque y se analizaron los conceptos de confusión y difusión.

1. Introducción

1.1 Planteamiento del problema

MiniDES utiliza una clave de únicamente 10 bits ($2^{10} = 1,024$ posibles combinaciones), lo cual lo hace vulnerable a ataques de fuerza bruta.

1.2 Contexto del problema

MiniDES permite comprender cómo funcionan cifrados reales como DES y AES, pero su tamaño reducido de clave lo vuelve inseguro.

1.3 Objetivos

- Implementar el ataque de fuerza bruta.
- Recuperar la clave utilizada.
- Analizar la complejidad $O(2^n)$.
- Comprender confusión y difusión.

1.4 Propuesta de solución

Se desarrolló un script que prueba las 1,024 claves posibles y compara el resultado hasta encontrar coincidencia.

2. Metodología y Teoría de Solución

El ataque consiste en probar todas las claves posibles (0-1023). La complejidad es $O(2^n)$, donde n es el tamaño de la clave.

Clave encontrada: 1001100111 (615 decimal)

Intentos: 616

Tiempo total: 0.46 segundos

3. Solución Propuesta

Se implementó `brute_force_attack.py` con análisis de tiempos y escalabilidad.

MiniDES (10 bits) $\rightarrow$ 1,024 claves $\rightarrow$ < 1 segundo

DES (56 bits) $\rightarrow 7.2 \times 10^{16}$ claves

AES-128 $\rightarrow 3.4 \times 10^{38}$ claves

AES-256 $\rightarrow 1.1 \times 10^{77}$ claves

4. Conclusión de resultados

- El tamaño de clave es crítico.
- MiniDES es inseguro por diseño.

- 128+ bits es estándar actual.

Conclusión general

La seguridad depende del esfuerzo computacional requerido para romper el sistema.

Apéndice

```
import des_e
import library_e
import time

def brute_force_attack(plaintext, ciphertext):
    """
    Realiza un ataque de fuerza bruta probando todas las claves posibles.

    Args:
        plaintext: El texto original en ASCII
        ciphertext: El texto cifrado en binario

    Returns:
        tuple: (clave_encontrada, tiempo_transcurrido, intentos)
    """
    print(f"\n{'='*70}")
    print(f"ATAQUE DE FUERZA BRUTA - MINIDES")
    print(f"{'='*70}")
    print(f"\nTexto original: {plaintext}")
    print(f"Texto cifrado: {ciphertext}")
    print(f"\nLa clave es de 10 bits, por lo que hay  $2^{10} = 1024$  claves posibles")
    print(f"\nIniciando búsqueda exhaustiva de la clave...\n")

    # Convertir el plaintext a binario para comparación
    plaintext_binary = library_e.text_to_bits(plaintext)

    # Tiempo de inicio
    start_time = time.time()

    # Probar todas las claves posibles (0 a 1023)
    for key_int in range(1024):
        # Convertir el número a binario de 10 bits
        key_binary = format(key_int, '010b')

        # Crear una instancia de DES con esta clave
        toy = des_e.DES()
```

```

toy.key = key_binary

# Intentar descifrar el ciphertext
entries = library_e.splitIntoGroups(ciphertext, 8)
decrypted_messages = []

try:
    for entry in entries:
        decryption = toy.Decryption(entry)
        decrypted_messages.append(decryption)

    decrypted_binary = "".join(decrypted_messages)

    # Comparar con el plaintext original
    if decrypted_binary == plaintext_binary:
        end_time = time.time()
        elapsed_time = end_time - start_time

        print(f"{'='*70}")
        print(f"¡CLAVE ENCONTRADA!")
        print(f"{'='*70}")
        print(f"Clave (binario): {key_binary}")
        print(f"Clave (decimal): {key_int}")
        print(f"Intentos: {key_int + 1}")
        print(f"Tiempo total: {elapsed_time:.6f} segundos")
        print(f"Tiempo promedio: {(elapsed_time/(key_int+1))*1000:.4f} ms por clave")
        print(f"{'='*70}\n")

        # Verificar descifrando el mensaje
        decrypted_text = library_e.text_from_bits(decrypted_binary)
        print(f"Verificación:")
        print(f"  Texto descifrado: {decrypted_text}")
        print(f"  Texto original: {plaintext}")
        print(f"  ¿Coinciden? {'SÍ ✓' if decrypted_text == plaintext else 'NO ✗'}\n")

        return key_binary, elapsed_time, key_int + 1

except:
    # Si hay un error en el descifrado, continuar con la siguiente clave
    pass

```

```

        # Mostrar progreso cada 100 intentos
        if (key_int + 1) % 100 == 0:
            print(f"Progreso: {key_int + 1}/1024 claves probadas
({(key_int+1)/1024*100:.1f}%)")

        # Si no se encuentra la clave
        end_time = time.time()
        elapsed_time = end_time - start_time
        print(f"\nNo se encontró la clave después de probar todas las 1024
posibilidades.")
        print(f"Tiempo total: {elapsed_time:.6f} segundos\n")

        return None, elapsed_time, 1024

def test_with_message(message):
    """
    Cifra un mensaje con la clave actual y luego realiza el ataque de
    fuerza bruta.

    Args:
        message: El mensaje a cifrar (en ASCII)
    """
    print(f"\n{'#' * 70}")
    print(f"PRUEBA CON MENSAJE: {message}")
    print(f"{'#' * 70}")

    # Cifrar el mensaje con la clave actual
    encrypted = library_e.encrypt(message)

    print(f"\nDetalles del cifrado:")
    print(f" Mensaje: {message}")
    print(f" Longitud: {len(message)} caracteres")
    print(f" Mensaje cifrado: {encrypted}")
    print(f" Longitud cifrada: {len(encrypted)} bits")

    # Obtener la clave actual del sistema
    toy = des_e.DES()
    original_key = toy.key
    print(f" Clave usada: {original_key}")

    # Realizar el ataque de fuerza bruta
    key_found, time_elapsed, attempts = brute_force_attack(message,
encrypted)

```

```

        if key_found:
            print(f"Resultado: Clave recuperada exitosamente en
{time_elapsed:.6f} segundos")
            if key_found == original_key:
                print(f"✓ La clave encontrada coincide con la clave original")
            else:
                print(f"X ADVERTENCIA: La clave encontrada ({key_found}) no
coincide con la original ({original_key}")
            else:
                print(f"Resultado: No se pudo recuperar la clave")

        return key_found, time_elapsed, attempts

def analyze_scalability():
    """
    Analiza la escalabilidad del ataque probando con diferentes longitudes
    de mensaje.
    """
    print(f"\n{'#' * 70}")
    print(f"ANÁLISIS DE ESCALABILIDAD")
    print(f"{'#' * 70}\n")

    test_cases = [
        "CARLOS",
        "CARLOS ALBERTO",
        "CARLOS ALBERTO LARA",
        "CARLOS ALBERTO LARA HERNANDEZ"
    ]

    results = []

    for test_message in test_cases:
        key, elapsed_time, attempts = test_with_message(test_message)
        results.append({
            'message': test_message,
            'length': len(test_message),
            'time': elapsed_time,
            'attempts': attempts,
            'key': key
        })
    print()

    # Resumen de resultados
    print(f"\n{'#' * 70}")

```

```

    print(f"RESUMEN DE RESULTADOS")
    print(f"{'='*70}\n")
    print(f"{'Mensaje':<35} {'Longitud':<10} {'Tiempo (s)':<15}
{'Intentos':<10}")
    print(f"{'-'*70}")

    for result in results:
        print(f"{'result['message']':<35} {'result['length']':<10}
{result['time']:<15.6f} {'result['attempts']:<10}")

    print(f"\n{'='*70}")
    print(f"ANÁLISIS:")
    print(f"{'='*70}")
    print(f"")
1. ESPACIO DE CLAVES:
    - MiniDES usa una clave de 10 bits
    - Total de claves posibles:  $2^{10} = 1,024$  claves
    - Es un espacio de claves extremadamente pequeño

2. TIEMPO DE ATAQUE:
    - Con hardware moderno, todas las claves se pueden probar en
milisegundos
    - El tiempo depende principalmente de la longitud del mensaje
    - Mensajes más largos requieren más operaciones de descifrado

3. ESCALABILIDAD:
    - El ataque de fuerza bruta es  $O(2^n)$  donde n es el tamaño de la clave
    - Para DES real (56 bits):  $2^{56} = 72,057,594,037,927,936$  claves
    - Para AES-128 (128 bits):  $2^{128} \approx 3.4 \times 10^{38}$  claves
    - Para AES-256 (256 bits):  $2^{256} \approx 1.1 \times 10^{77}$  claves

4. CONCLUSIÓN:
    - MiniDES es solo educativo, completamente inseguro para uso real
    - Demuestra la importancia del tamaño de clave en criptografía
    - Un cifrado moderno necesita al menos 128 bits para ser seguro
    "")

if __name__ == "__main__":
    print("\n" + "="*70)
    print(" "*15 + "PRÁCTICA MINIDES - ATAQUE DE FUERZA BRUTA")
    print(" "*20 + "Carlos Alberto Lara Hernández")
    print("="*70)

# Actividad 1.3: Enviar el nombre y observar bytes cifrados

```



```
print("\n\n--- ACTIVIDAD 1.3: CIFRADO DEL NOMBRE ---\n")
nombre = "CARLOS ALBERTO LARA HERNANDEZ"
encrypted_nombre = library_e.encrypt(nombre)
print(f"Nombre:           {nombre}")
print(f"Bytes cifrados:      {encrypted_nombre}")
print(f"Longitud:            {len(encrypted_nombre)} bits")

# Actividad 1.4 y 2.1: Probar todas las claves posibles y recuperar la
clave correcta
print("\n\n--- ACTIVIDAD 1.4 y 2.1: ATAQUE DE FUERZA BRUTA ---\n")
key_found1, time1, attempts1 = test_with_message("CARLOS ALBERTO LARA
HERNANDEZ")

# Actividad 2.2: Probar con nombre más largo
print("\n\n--- ACTIVIDAD 2.2: NOMBRE COMPLETO MÁS LARGO ---\n")
key_found2, time2, attempts2 = test_with_message("CARLOS ALBERTO LARA
HERNANDEZ DE LA CIUDAD")

# Actividad 2.3: Análisis de escalabilidad
print("\n\n--- ACTIVIDAD 2.3: ANÁLISIS DE ESCALABILIDAD ---\n")
analyze_scalability()

print("\n\n" + "="*70)
print("  "*20 + "FIN DEL ANÁLISIS")
print("="*70 + "\n")
```